01 The problem

- Question answering over a fixed set of ECE notes, where the answer must be **grounded and cited**.
- Some questions need a **tool** (a calculation, a formula), some are small talk, some the notes cannot answer at all.
- Must run **with no API key** so it and its evaluation are reproducible, and upgrade to a hosted model when a key exists.
- The interesting part is the **process**: route, retrieve, grade, retry - all visible.

4.1 Retrieval

```
PDF → PyPDFLoader → split(800/150) → all-MiniLM-L6-v2 (384-d,
```

8 documents · 27 chunks

mean chunk 652 chars; generated corpus, fully checkable

Exact cosine

IndexFlatIP over normalised vectors - the score in every trace is a real cosine

Local & offline

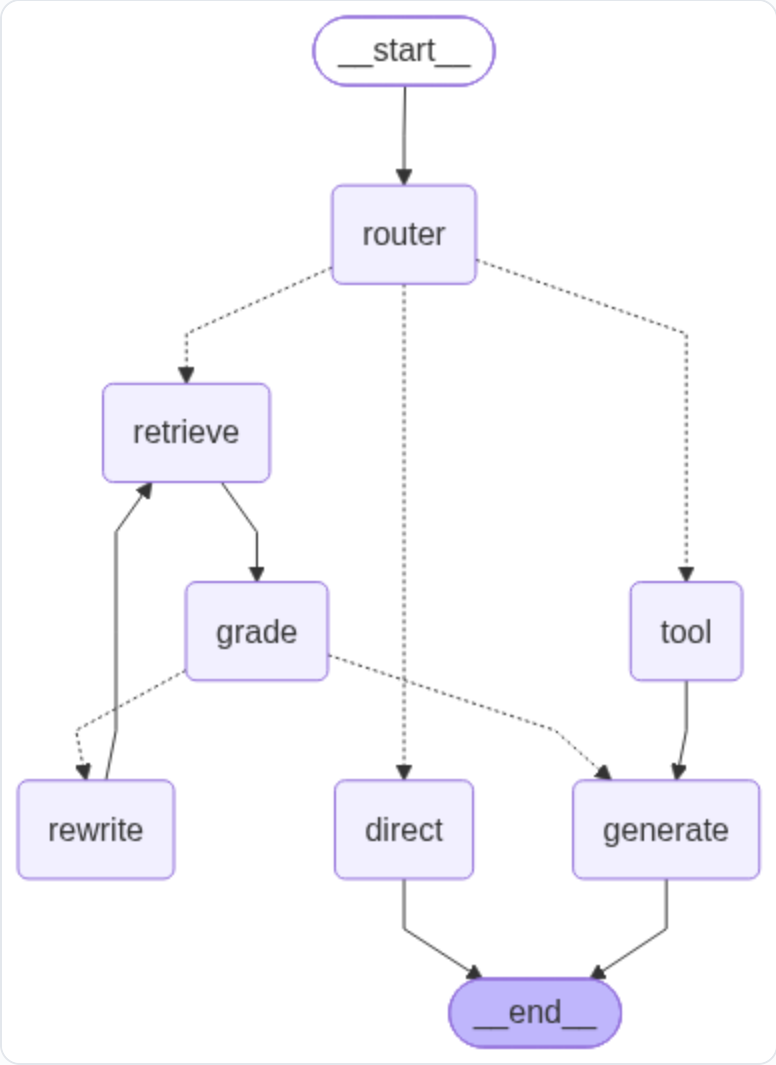
no key, no cost; Unicode-safe FAISS persistence in store.py

4.2 The agent is a graph, not a chain

```
START → router -+→ retrieve → grade -+→ generate → END
                |                    +→ rewrite → retrieve
                +→ tool ———→ generate → END
                +→ direct --→ END
```

- Conditional **and cyclic**: routing is runtime; a failed grade rewrites and re-retrieves.
- `AgentLLM` exposes `route/grade/rewrite/generate` - the graph never knows if a hosted model or the offline scorer answers.

4.2 Workflow diagram



Rendered from the compiled graph itself, so it cannot drift from the code.

4.2 Tools, kept offline and boring

calculator

Numeric evaluation via a **restricted AST walk** - never `eval`, because tool input is untrusted model output. Handles `pi`, `e`, `^`.

formula_lookup

A curated, sourced table of canonical ECE formulas (Nyquist, Shannon, Carson, VSWR, ...). Cites the notes; keeps the evaluation reproducible.

4.3 Evaluation - 22 labelled questions

Metric	offline	LLM (Groq)
routing accuracy	**100%**	**100%**
retrieval hit rate	**100%**	**100%**
answer keyword rate	**93%**	**100%**
tool success rate	**100%**	**100%**
out-of-scope refusal	**67%**	**100%**
latency (median)	56 ms	4.4 s

The 100%s are a ceiling from a small, self-authored set; the hosted LLM fixes the two weak (informative) metrics. Both runs are traced in LangSmith and replayed through the `week4-study-assistant-eval` dataset with evaluators.

4.3 The two real failures

q20 - Friis equation (refusal 67%)

Not in the corpus, but "transmission ... formula" makes retrieval return the transmission-lines notes and lexical grading passes it. Result: a fluent, **cited, wrong** answer. Root cause - overlap cannot separate "on topic" from "answers it".

q22 - FM bandwidth (keyword 93%)

Retrieval was **correct**; the extractive generator quoted the neighbouring sentence instead of Carson's rule. A generation-ranking failure, not a retrieval one.

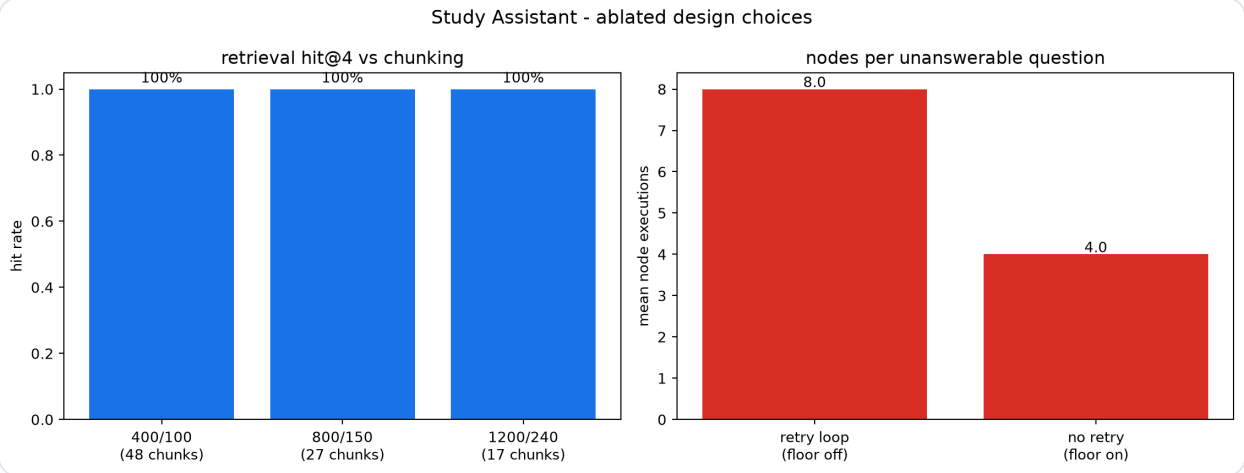
4.3 Ablations - decisions, measured

Chunk size

hit@4 = 100% at 400/800/1200. Smaller chunks sharpen top-1 cosine (0.593 vs 0.517) at ~3x the index size. 800/150 kept as a non-degenerate default.

Retry floor

Low-coverage floor skips hopeless retries: **8 -> 4 nodes**, **49 -> 22 ms** per unanswerable question, same correctness.



4.3 Traces

Every node records a span. The offline run renders a local waterfall (`eval/local_tracer.py`); both configurations also stream 22 tagged trees to **LangSmith**, and `docs/langsmith_figures.py` pulls the real waterfalls back from the API.

Grounded · tool-only · refusal · corrective-retry loop · plus an offline-vs-LLM dataset experiment with custom evaluators.

4.4 Demo

- Streamlit chat: answer, confidence badge, citations.
- Full **agent trace + retrieved chunks** expandable under every answer.
- Sidebar reports the live backend, embeddings, retrieval settings and index stats.

```
streamlit run src/app.py
```

05 Limitations & next

- Small self-authored corpus/test set; lexical grader; extractive default answers; single-run numbers; the hosted pass used `gpt-oss-20b` not the configured `gpt-oss-120b`.
- **Next:** traced `gpt-oss-120b` pass → cross-encoder reranker (q22) → bigger corpus → web-search tool.

Everything runs with no API key; set `GROQ_API_KEY` / `LANGSMITH_API_KEY` to upgrade.